

AADK 9: Task 3 — Feature Implementation

Strategy: Conditionals + Nullability Integration

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 9

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Feature Definition

Feature: Event RSVP Screen

The **Event RSVP screen** allows users to respond to an event invitation. The user can enter the **number of guests**, choose a **meal preference**, and toggle whether they **accept or decline the invitation**. Based on the inputs, the system decides whether to enable the **Submit button**, show validation errors, or display a confirmation message.

The feature must handle **nullable inputs**, such as blank guest fields or missing meal preferences, while ensuring the UI reacts correctly to user decisions.

2. Data & State Model

Data Fields

Field Name	Type	Nullable	Purpose
guests	Int?	Yes	Number of guests entered
mealPreference	String?	Yes	Selected meal option
rsvpStatus	Boolean	No	Accept or Decline response

Compose State Variables

```
var guestsInput by remember { mutableStateOf("") }
```

```
var mealPreferenceInput by remember { mutableStateOf<String?>(null) }
```

```
var rsvpStatus by remember { mutableStateOf(false) }
```

Convert guest input to nullable integer:

```
val guestsCount = guestsInput.toIntOrNull()
```

Conditions & State Transitions

Condition	Result
guestsInput blank	guestsCount = null
guestsCount <= 0	show error
RSVP declined	show "Thank you for responding"
mealPreference null	default to "Standard"
RSVP accepted AND guests > 0	enable Submit button

3. Pseudo-Code / Skeleton Logic

Core Logic

```
val guestsCount = guestsInput.toIntOrNull()
```

```
val mealPreference = mealPreferenceInput ?: "Standard"
```

```
if (rsvpStatus) {
```

```
    if (guestsCount != null && guestsCount > 0) {
```

```
        // enable submit button
```

```
    } else {
```

```
        // show error message
```

```
    }
```

```

} else {
    // user declined invitation
    // show "Thank you for your response"
}

```

Submit Button Logic

```
val isEnabled = RSVPStatus && (guestsCount ?: 0) > 0
```

Example Compose Snippet

```

Button(
    onClick = { /* submit logic */ },
    enabled = RSVPStatus && (guestsInput.toIntOrNull() ?: 0) > 0
) {
    Text("Submit RSVP")
}

```

4. UI Sketch & Conditions Mapping

UI Layout Sketch

	Event	RSVP	
Guests:	[	Input	]
Meal:	[	Dropdown ▼	]
Accept Invitation	[	Toggle	]
[	Submit RSVP	]	
Result	Message		

Compose Components Mapping

UI Component	Purpose	Null Handling
TextField	Enter guest count	Parse using toIntOrNull()
DropDownMenu	Select meal preference	Default "Standard"
Switch / Toggle	Accept or decline RSVP	Boolean state
Button	Submit RSVP	Enabled only if valid input
Text	Display messages	Shows errors or confirmation

Example Compose UI Handling

```
TextField(  
    value = guestsInput,  
    onValueChange = { guestsInput = it },  
    label = { Text("Number of Guests") }  
)  
  
Text(  
    text = mealPreferenceInput ?: "Standard Meal Selected"  
)  
  
if (!rsvpStatus) {  
    Text("Thank you for your response.")  
}
```

5. Edge-Case & Test Planning

Edge Case	Expected UI Outcome	Logic Handling
Guests field blank	Submit disabled	guestsCount becomes null
Guests = 0	Error message	Check guestsCount > 0
Meal preference null	Default meal shown	Elvis operator ?:
Rapid toggle changes	UI updates instantly	Compose state recomposition

6. Reflection

This feature was more complex because it required combining **conditional logic with nullable data handling** while also ensuring the UI updates correctly. Managing text input conversion to nullable integers added additional validation complexity. Combining conditionals and nullability increases cognitive load because developers must consider multiple possible states and edge cases. To reduce complexity, I used strategies such as **default values (Elvis operator)**, **clear state variables**, and **simplified condition checks**. Planning the logic beforehand helped ensure the feature remains readable, maintainable, and robust.